PDF ① 필수 용어·핵심 지식 암기집 (Terms & Core Rules)

대상 범위: SSAFY DB — SQL 01 (153p) · SQL 02 (113p) · Many To One Relationships 01 (81p) · Many To One Relationships 02 (76p)

설계 목적: 단순 한 줄 사전이 아니라, 실전 쿼리·ORM 작성 시 즉각 떠올려야 할 정확한 표기·사용 조건·비교 기준·반례를 체화하는 암기 교본

0. 학습 안내 및 전체 핵심 용어 인덱스 (30대 필수 항목)

본 암기집은 업로드된 원본 교재 4권(총 423페이지)에서 **이후 학습의 선행 조건이 되는 용어**, **실습·시험에서 반복 등장하는 문법·공식**, **비슷해서 혼동하기 쉬운 개념 쌍**을 엄선하여 5개 파트로 재구성하였다. 각 항목은 ① 원어/정확한 표기, ② 한국어 명칭, ③ 형성 배경/맥락, ④ 정확한 정의, ⑤ 초보자 눈높이 설명, ⑥ 실제 사용 상황, ⑦ 적용 예시(코드/수식), ⑧ 실행 결과, ⑨ 중요도·혼동 주의점·반례·회상 질문, ⑩ 원본 출처 위치를 빠짐없이 수록하였다.

개념 ID	원어 / 정확한 표기	한국어 명칭	원본 출처	기억할 핵심 구별점 (1줄 요약)
TERM-01	Database (DB) & CRUD	데이터베이스와 기본 조작	db 1 (p.8~17)	스프레드시트 한계(100만 행·보안·갱신 누락) 극복, 생성·조회·수정·삭제
TERM-02	Table / Field / Record / Schema	테이블·필드·레코드·스키마	db 1 (p.26~29)	Table(Relation)=표, Field(Column/Attribute)=열(타입), Record(Row/Tuple)=행(값)
TERM-03	Primary Key (PK) vs Foreign Key (FK)	기본 키와 외래 키	db 1 (p.23~31)	PK는 행 고유 식별자(중복·NULL 불가), FK는 타 테이블 PK 참조 연결고리
TERM-04	RDBMS & SQLite	관계형 DB 관리 시스템	db 1 (p.33~37)	테이블 간 관계를 관리하는 SW; SQLite는 무설치·파일 내장형 경량 RDBMS
TERM-05	DDL / DQL / DML / DCL	SQL 4대 문장 유형	db 1 (p.44~50)	DDL(구조 정의), DQL(SELECT 검색), DML(INSERT·UPDATE·DELETE 조작), DCL(권한)
TERM-06	SELECT ... FROM & AS / 산술 연산	데이터 조회와 별칭·나눗셈	db 1 (p.51~63)	정수/정수 나눗셈은 소수점 버림! 소수점 유지 시 `60000.0`으로 나눌 것
TERM-07	ORDER BY (ASC / DESC) & NULL 정렬	결과 정렬과 NULL 위치	db 1 (p.65~76)	기본값은 ASC(오름차순); SQLite는 NULL을 가장 작은 값으로 취급하여 ASC 맨 앞 배치
TERM-08	DISTINCT	중복 레코드 제거 키워드	db 1 (p.79~84)	반드시 SELECT 키워드 바로 뒤에 작성하며 지정 컬럼 조합의 고유값만 반환

개념 ID	원어 / 정확한 표기	한국어 명칭	원본 출처	기억할 핵심 구별점 (1줄 요약)
TERM-09	WHERE & BETWEEN / IN / NOT IN	행 조건 필터링과 범위·목록 연산자	db 1 (p.85~102)	GROUP BY 이전에 개별 행 필터링; BETWEEN은 양 끝값 포함(이상·이하)
TERM-10	LIKE & Wildcards (%, _)	패턴 일치 연산자와 와일드카드	db 1 (p.103~109)	%는 0개 이상 임의 문자열, _는 정확히 단일 문자(1글자)와 일치
TERM-11	NULL & 3-Valued Logic / IS NULL	3값 논리와 NULL 명시적 비교	db 1 (p.91, 135~138)	`NULL = NULL`은 TRUE가 아니라 UNKNOWN! 반드시 `IS NULL` 사용
TERM-12	LIMIT row_count OFFSET offset	조회 개수 제한 및 시작 위치 지정	db 1 (p.110~116)	OFFSET은 0부터 시작(건너뛸 개수); "4번째~7번째"는 `LIMIT 4 OFFSET 3`
TERM-13	GROUP BY & Aggregate Functions	그룹화와 5대 집계 함수	db 1 (p.118~125)	`SUM, AVG, MAX, MIN, COUNT`; 그룹별 요약 행 생성, AS 별칭으로 정렬 가능
TERM-14	HAVING vs WHERE & 7단계 실행 순서	그룹 조건절과 SELECT 실행 파이프라인	db 1 (p.126~131)	WHERE에 집계함수 쓰면 에러 (`misuse of aggregate`); `FROM→WHERE→GROUP BY→HAVING→SELECT→ORDER BY→LIMIT`
TERM-15	CREATE TABLE & PRAGMA table_info()	테이블 생성과 메타데이터 구조 확인	db 2 (p.9~15)	컬럼명·데이터타입·제약조건 선언; `PRAGMA table_info('테이블명');`으로 cid·pk 확인
TERM-16	SQLite Data Types & Type Affinity	5대 스토리지 클래스와 타입 선호도	db 2 (p.16, 94~100)	`NULL, INTEGER, REAL, TEXT, BLOB`; `VARCHAR(50)`도 TEXT로 추론·길이 미검사, 날짜는 TEXT/REAL/INTEGER로 저장
TERM-17	Constraints & AUTOINCREMENT	제약 조건과 자동 증가 속성	db 2 (p.17~19, 97~98)	반드시 `INTEGER PRIMARY KEY AUTOINCREMENT`로만 선언(`INT` 에러), 삭제된 번호 재사용 불가
TERM-18	ALTER TABLE (4종) & DROP TABLE	테이블 구조 변경과 테이블 삭제	db 2 (p.20~39)	`ADD COLUMN` 시 NOT NULL이면 `DEFAULT` 필수! 단일 문 다중 컬럼 추가 불가
TERM-19	INSERT / UPDATE / DELETE & Subquery	데이터 삽입·수정·삭제와 서브쿼리	db 2 (p.42~67)	UPDATE·DELETE에서 WHERE 생략 시 전체 행 변경/삭제; 오래된 2개 삭제는 `WHERE id IN (SELECT ... LIMIT 2)`
TERM-20	INNER JOIN vs LEFT JOIN (+ IS NULL)	내부 조인과 왼쪽 외부 조인	db 2 (p.68~92)	INNER는 양쪽 일치 행만, LEFT는 왼쪽 전체 보존(불일치 시 NULL); `LEFT JOIN + IS NULL`은 "~한 적 없는" 패턴

개념 ID	원어 / 정확한 표기	한국어 명칭	원본 출처	기억할 핵심 구별점 (1줄 요약)
TERM-21	Many-to-One (N:1) & ForeignKey 위치	다대일 관계와 외래 키 배치 원칙	db 3 (p.8~15)	FK는 반드시 **N(다수, Comment)** 테이블에 배치! 1(Article)에 두면 데이터 중복 발생
TERM-22	models.ForeignKey(to, on_delete)	Django 외래 키 클래스와 삭제 옵션	db 3 (p.17~21, 64)	to와 on_delete 필수(CASCADE, PROTECT, SET_NULL); DB에는 필드명_id(bigint)로 생성
TERM-23	Reference (참조) vs Reverse Reference (역참조)	정방향 참조와 역참조 매니저	db 3 (p.25~40)	참조(N→1): comment.article / 역참조(1→N): article.comment_set.all()
TERM-24	form.save(commit=False)	DB 즉시 저장 보류 옵션	db 3 (p.43~54)	ModelForm에서 제외한 FK(article, user)를 인스턴스에 주입한 뒤 최종 save() 호출
TERM-25	DTL {% for ... %}{% empty %} & count()	빈 목록 대체 출력과 개수 집계	db 3 (p.68~70)	반복 요소 없을 때 {% empty %} 실행; {{ comments|length }} 또는 {{ article.comment_set.count }}
TERM-26	settings.AUTH_USER_MODEL vs get_user_model()	User 모델 참조 이원화와 지연 평가	db 4 (p.8~12)	models.py는 문자열 반환 settings.AUTH_USER_MODEL(지연 평가), 그 외 파일은 get_user_model()
TERM-27	Ownership Authorization (request.user)	작성자 본인 확인 및 이중 권한 제어	db 4 (p.23~26, 39~40)	템플릿(버튼 숨김) + 뷰 함수(if request.user == article.user:) 양쪽 모두 검증 필수(Postman 우회 방어)
TERM-28	View Decorators & HTTP 405	뷰 데코레이터와 허용 메서드 제한	db 4 (p.41~49, 65)	@login_required, @require_http_methods, @require_safe(GET+HEAD 허용 권장), @require_POST; 위반 시 405
TERM-29	ERD (Entity-Relationship Diagram)	개체-관계 다이어그램 3대 구성요소	db 4 (p.50~56, 60~62)	Entity(개체=테이블), Attribute(속성=컬럼), Relationship(관계=외래키 연결선)
TERM-30	Cardinality & Crow's Foot Notation	수적 관계와 까마귀 발 표기법 6종	db 4 (p.57~59)	O(0, 선택), |(1, 필수), 까마귀 발 <(Many, 다수)의 조합으로 6가지 수적 관계 표현

PART 1. 데이터베이스 기초 · RDBMS · SQL 문법 체계 (db 1.pdf 전반부)

[TERM-01] Database (DB) & CRUD — 데이터베이스와 4대 기본 조작

- ① **원어·정확한 표기** / ② **한국어 명칭**: Database (DB) / 데이터베이스, 그리고 CRUD (Create, Read, Update, Delete / 생성, 조회, 수정, 삭제)
- ③ **배경 및 맥락**: 텍스트 파일(개인정보.txt)은 구조적 관리가 불가능하고, 스프레드시트(엑셀)는 행/열 구조를 제공하지만 **① 크기(일반적으로 약 100만 행까지만 저장 가능)**, **② 보안(파일/링크 소유 여부에 따른 단순 접근 권한)**, **③ 정확성(예: '강원'이 '강언'으로 바뀔 때 분산된 여러 시트를 일일이 수정하다 누락 발생)**이라는 3대 한계를 가진다. 이를 해결하기 위해 체계적인 데이터베이스가 등장했다.
- ④ **정확한 정의**: **데이터(Data)**는 저장이나 처리를 위해 컴퓨터가 다룰 수 있도록 변환·구조화된 정보이며, **데이터베이스(Database)**는 체계적으로 정리된 데이터의 모음으로서 데이터를 구조적으로 저장하고 기본 조작(CRUD)을 수행하는 역할이다.
- ⑤ **초보자를 위한 쉬운 설명**: 책을 바닥에 아무렇게나 쌓아두면(일반 파일) 찾기 어렵지만, 주제별·작가별 번호표를 붙여 서가에 정리해 두면(데이터베이스) 수백만 권 중에서도 원하는 책을 즉시 꺼내고(Read), 새 책을 꽂고(Create), 표지를 바꾸고(Update), 폐기(Delete)할 수 있는 것과 같다.
- ⑥ **실제 사용 상황** / ⑦ **적용 예시** / ⑧ **결과**: 배달의민족 연간 11억 건 이상 주문 처리, 유튜브 1분당 500시간 동영상 업로드 관리 등 대규모 서비스에서 데이터를 중복 없이 나누어 저장하고 CRUD SQL 문장으로 제어한다.
- ⑨ **중요도 [★★★] · 혼동 주의점 · 회상 질문**:
  - *혼동 주의(반례)*: "엑셀도 표 형태니까 관계형 데이터베이스다" → **거짓(X)**. 스프레드시트는 100만 행 제한, 세밀한 권한 제어 불가, 데이터 중복으로 인한 갱신 누락 문제가 있어 RDB를 대체할 수 없다.
  - *회상 질문*: 스프레드시트 대신 데이터베이스를 사용해야 하는 3가지 한계 기준은 무엇인가? (정답: 크기, 보안, 정확성)
- ⑩ **원본 출처**: db 1.pdf p.8~17, p.148, p.150

[TERM-02] Table (Relation) / Field (Column, Attribute) / Record (Row, Tuple) / Database (Schema)

- ① **원어·정확한 표기** / ② **한국어 명칭**:
  - **Table** (aka *Relation*) — 테이블 (릴레이션)
  - **Field** (aka *Column, Attribute*) — 필드 (컬럼, 열, 속성)
  - **Record** (aka *Row, Tuple*) — 레코드 (로우, 행, 튜플)
  - **Database** (aka *Schema*) — 데이터베이스 (스키마: 테이블들의 집합)
- ③ **배경 및 맥락**: 관계형 데이터베이스 이론(수학적 집합론)에서 유래한 학술 용어(Relation, Attribute, Tuple)와 실무 SQL 용어(Table, Column, Row), 모델링 용어(Table, Field, Record)가 현업과 시험에서 혼용되므로 3쌍의 동의어를 정확히 매칭해야 한다.
- ④ **정확한 정의** / ⑤ **쉬운 설명**:
  - **Table**: 데이터를 기록하는 표 전체 공간. 여러 Table이 모인 집합을 **Database(Schema)**라 한다.
  - **Field (Column, Attribute)**: 테이블의 세로 방향 항목. 각 필드에는 고유한 데이터 형식(타입, 예: 숫자, 문자열)이 지정된다.
  - **Record (Row, Tuple)**: 테이블의 가로 한 줄. 한 대상(예: 고객 1명)에 대한 구체적인 데이터 값들의 묶음이다.
- ⑥ **실제 사용 상황** / ⑦ **적용 예시** / ⑧ **결과**: 고객 테이블에서 세로줄 id, 이름, 청구지, 주소지는 4개의 **Field(열)**이고, 가로줄 (2, '유미선', '강원', '경기') 한 줄은 1개의 **Record(행)**이다.
- ⑨ **중요도 [★★★] · 혼동 주의점 · 회상 질문**:

  - *핵심 구별점*: "고유한 데이터 타입(INTEGER, VARCHAR 등)이 지정되는 단위"를 물으면 **Field(Column)**이고, "구체적인 데이터 값들의 묶음(고유 식별 대상)"을 물으면 **Record(Row)**이다.
  - *회상 질문*: Field와 동의어인 2개 영단어, Record와 동의어인 2개 영단어는? (정답: Field = Column, Attribute / Record = Row, Tuple)

- ⑩ **원본 출처**: `db 1.pdf` p.26~29, p.148

[TERM-03] Primary Key (PK, 기본 키) vs Foreign Key (FK, 외래 키)

- ① **원어·정확한 표기** / ② **한국어 명칭**: Primary Key (PK, 기본 키) / Foreign Key (FK, 외래 키)
- ③ **배경 및 맥락**: 병원 접수창구나 쇼핑몰에서 동명이인('김한웅'이 2명)이거나 주소지가 같을 때, 이름이나 주소만으로는 누가 어떤 진료/주문을 했는지 식별할 수 없다. 이를 해결하기 위해 주민등록번호처럼 절대 겹치지 않는 고유 번호(PK)를 부여하고, 다른 표에서 그 번호를 가리키게(FK) 만들었다.
- ④ **정확한 정의**:
  - **Primary Key (기본 키, PK)**: 각 레코드(행)를 유일하게 식별하는 고유한 값. 중복된 값이나 `NULL`을 절대 허용하지 않는다.
  - **Foreign Key (외래 키, FK)**: 한 테이블의 필드 중 다른 테이블의 레코드를 식별할 수 있는 키. 다른 테이블의 **기본 키(PK)를 참조**하여 두 테이블 간의 논리적 관계(Relationship)를 만든다.
- ⑤ **쉬운 설명** / ⑥ **사용 상황** / ⑦ **적용 예시** / ⑧ **결과**: `고객` 테이블의 `id`(1: 김한웅, 2: 유미선, 3: 박지수)는 각 고객을 구분하는 **PK**다. `주문` 테이블에 `고객 ID`라는 필드를 만들어 `3`을 적어두면, 이 `고객 ID`가 **FK**가 되어 1번 주문과 2번 주문 모두 3번 고객(박지수)이 주문했다는 사실을 중복 없이 정확히 연결한다.
- ⑨ **중요도 [★★★]** · **혼동 주의점** · **회상 질문**:
  - *혼동 주의(반례)*: "외래 키(FK)도 기본 키처럼 한 테이블 안에서 값이 중복될 수 없다" → **거짓(X)**. 박지수(고객 ID=3)가 주문을 여러 번 하면 `주문` 테이블의 외래 키(`고객 ID`)에는 `3`이 여러 행에 중복 등장한다!
  - *회상 질문*: 기본 키(PK)가 절대 가질 수 없는 두 가지 상태는? (정답: 중복된 값, NULL 값)
- ⑩ **원본 출처**: `db 1.pdf` p.18~25, p.30~31, p.38, p.144, p.148 / `db 2.pdf` p.104

[TERM-04] DBMS / RDBMS / SQLite

- ① **원어·정확한 표기** / ② **한국어 명칭**: DBMS (Database Management System, 데이터베이스 관리 시스템) / RDBMS (Relational Database Management System, 관계형 데이터베이스 관리 시스템) / SQLite
- ③ **배경 및 맥락** / ④ **정확한 정의**:
  - **DBMS**: 데이터베이스를 관리하는 소프트웨어 프로그램. 데이터베이스와 사용자 간의 인터페이스 역할을 하며 데이터 구성, 업데이트, 모니터링, 백업, 복구를 돕는다.
  - **RDBMS**: 관계형 모델을 기반으로 여러 테이블에 데이터를 나누어 저장하되 공통 키(PK/FK)로 관계를 맺고 함께 관리하는 소프트웨어 프로그램. (종류: `SQLite`, `MySQL`, `PostgreSQL`, `Oracle Database` 등)
  - **SQLite**: 별도 서버 설치 없이 가볍게 실행 가능한 경량 오픈소스 RDBMS. 컴퓨터나 모바일 기기에 파일 단위로 내장(Embedded)되어 동작하므로 모바일 앱이나 소규모 프로그램, 학습 실습 환경(`sample.sqlite3`)에 적합하다.
- ⑨ **중요도 [★★☆]** · **원본 출처**: `db 1.pdf` p.32~38, p.145, p.150

[TERM-05] SQL & SQL Statements 4대 유형 (DDL / DQL / DML / DCL)

- ① **원어·정확한 표기** / ② **한국어 명칭**: SQL (Structured Query Language, 구조화된 질의 언어) 및 수행 목적에 따른 4대 Statement 유형
- ③ **배경 및 맥락** / ④ **정확한 정의**: **SQL**은 테이블 형태로 구조화된 관계형 데이터베이스에게 데이터 조회 및 조작을 요청(질의, Query)하는 표준 언어다(ANSI/ISO 표준 채택). SQL을 구성하는 기본적인 코드 블록을 **SQL Statement(문장)**라

하며, 각 문장 끝에는 마침표 역할인 세미콜론(;)을 찍는다. SQL 키워드는 대소문자를 구분하지 않으나 명시적 구분을 위해 대문자 작성을 권장한다.

- **⑥ 실제 사용 상황 / ⑦ 적용 예시 (4대 유형 핵심 구분표)**:

유형 약어	원어 전체 표기	핵심 역할	소속 SQL 키워드 (필수 암기)
DDL	Data **Definition** Language (데이터 정의 언어)	데이터의 기본 **구조(테이블·스키마)** 및 형식 정의·변경·삭제	CREATE, DROP, ALTER
DQL	Data **Query** Language (데이터 검색/질의 언어)	테이블에서 원하는 조건의 **데이터 검색(조회)**	SELECT
DML	Data **Manipulation** Language (데이터 조작 언어)	테이블 내부의 **데이터(행/레코드) 조작** (추가, 수정, 삭제)	INSERT, UPDATE, DELETE
DCL	Data **Control** Language (데이터 제어 언어)	데이터 및 작업(트랜잭션)에 대한 **사용자 권한·반영 제어**	COMMIT, ROLLBACK, GRANT, REVOKE

- **⑨ 중요도 [★★★] · 혼동 주의점 · 회상 질문**:
  - *가장 많이 틀리는 함정*: DROP(DDL: 테이블 통째로 철거) vs DELETE(DML: 테이블 안의 행만 삭제), 그리고 ALTER(DDL: 컬럼 구조 변경) vs UPDATE(DML: 행 내부의 데이터 값 수정)를 섞어 선지로 출제한다!
  - *회상 질문*: 테이블에 새 컬럼을 추가하는 ALTER TABLE은 DDL인가 DML인가? (정답: 구조를 바꾸므로 DDL)
- **⑩ 원본 출처**: db 1.pdf p.39~45, p.50, p.133~134, p.139~140 / db 2.pdf p.7, p.41

PART 2. DQL 데이터 조회 · 정렬 · 필터링 · 집계 (db 1.pdf 후반부)

[TERM-06] SELECT ... FROM / Asterisk(*) / AS 별칭 / 정수 나눗셈 규칙

- **① 원어·정확한 표기 / ② 한국어 명칭**: SELECT select_list FROM table_name; / * (Asterisk, 전체 필드) / AS '별칭' (Alias)
- **④ 정확한 정의 / ⑤ 쉬운 설명**: FROM 뒤에 조회할 테이블 이름을 적고, SELECT 뒤에 가져올 필드 목록을 쉼표(,)로 나열한다. 모든 필드를 가져올 때는 *를 쓰며, 출력 컬럼명을 바꿀 때는 필드명 AS '별칭'을 사용한다.
- **⑥ 실제 사용 상황 / ⑦ 적용 예시 / ⑧ 결과**:

```
SELECT Name, Milliseconds / 60000 AS '재생 시간(분)' FROM tracks;
```

결과: Milliseconds가 343719일 때 5.728...이 아니라 소수점이 버려진 정수 5가 출력된다.

- **⑨ 중요도 [★★★] · 혼동 주의점 · 반례**:
  - *핵심 규칙*: SQL(SQLite)에서 **정수끼리 나누면(Milliseconds / 60000) 소수점 이하가 무조건 버려진다!** 소수점까지 정확히 보려면 반드시 실수 리터럴인 **60000.0**으로 나누어야 한다.
- **⑩ 원본 출처**: db 1.pdf p.51~63, p.142, p.149

[TERM-07] ORDER BY (ASC / DESC) & 다중 정렬 · NULL 정렬 위치

- **① 원어·정확한 표기 / ② 한국어 명칭**: ORDER BY column1 [ASC|DESC], column2 [ASC|DESC]; (조회 결과 정렬 절)
- **④ 정확한 정의**: FROM(및 WHERE, GROUP BY) 뒤에 위치하여 하나 이상의 컬럼을 기준으로 결과를 오름차순(ASC, 기본값—생략 가능) 또는 내림차순(DESC)으로 정렬한다.
- **⑥ 실제 사용 상황 / ⑦ 적용 예시 / ⑧ 결과**:

```
SELECT Country, City FROM customers ORDER BY Country DESC, City;
```

결과: 먼저 Country를 기준으로 알파벳 역순(United Kingdom → USA ...) 정렬한 뒤, **같은 Country 그룹 내부에서만** 2차 기준인 City를 오름차순(Boston → Chicago → Cupertino ...)으로 정렬한다.

- ⑨ **중요도 [★★★] · 혼동 주의점 · 반례**:
  - *NULL 정렬 규칙*: 정렬 대상 컬럼(예: ReportsTo)에 NULL 값이 존재할 때, **SQLite는 NULL을 가장 작은 값으로 취급**한다. 따라서 오름차순(ASC) 정렬 시 결과의 **맨 위(가장 먼저)**에 NULL이 출력된다(DBMS마다 다를 수 있음).
  - *반례*: ORDER BY Country, City DESC;라고 쓰면 두 컬럼 모두 내림차순이 되는가? → **아니다(X)**. Country에는 DESC가 없으므로 기본값인 ASC(오름차순)가 적용되고 City만 내림차순이 된다.
- ⑩ **원본 출처**: db 1.pdf p.65~76, p.143

[TERM-08] Clause vs Operator & DISTINCT (중복 제거)

- ① **원어·정확한 표기** / ② **한국어 명칭**: Clause(절: DISTINCT, WHERE, LIMIT 등 문장 구성 요소) vs Operator(연산자: BETWEEN, IN, LIKE, 비교/논리 기호) / SELECT DISTINCT select_list FROM table_name;
- ④ **정확한 정의** / ⑦ **적용 예시** / ⑧ **결과**: DISTINCT는 조회 결과에서 중복된 레코드를 제거하여 고유한 값만 반환한다. **반드시 SELECT 키워드 바로 뒤에 작성**해야 한다.

```
SELECT DISTINCT Country FROM customers ORDER BY Country;
```

결과: Brazil이 5번, Canada가 4번 있어도 결과에는 각각 1번씩만 출력된다.

- ⑨ **중요도 [★★☆] · 원본 출처**: db 1.pdf p.78~84, p.143

[TERM-09] WHERE & Comparison / Logical Operators (BETWEEN, IN, NOT IN)

- ① **원어·정확한 표기** / ② **한국어 명칭**: WHERE search_condition / 비교 연산자(=, >=, <=, !=, IS, LIKE, IN, BETWEEN...AND) / 논리 연산자(AND, OR, NOT)
- ④ **정확한 정의** / ⑦ **적용 예시**:
  - Bytes BETWEEN 100000 AND 500000: Bytes >= 100000 AND Bytes <= 500000과 완전히 동일하다(양 끝 경계값 포함).
  - Country IN ('Canada', 'Germany', 'France'): Country = 'Canada' OR Country = 'Germany' OR Country = 'France'의 축약형이다. 제외할 때는 NOT IN (...)을 사용한다.
- ⑨ **중요도 [★★★] · 혼동 주의점(반례)**: WHERE Country = 'Canada' OR 'Germany'처럼 컬럼명을 생략하고 값만 OR로 나열하면 의도대로 동작하지 않는다! OR를 쓸 때는 매번 Country = ...을 반복하거나 IN (...)을 써야 한다.
- ⑩ **원본 출처**: db 1.pdf p.85~102, p.108

[TERM-10] LIKE Operator & Wildcard Characters (%, _)

- ① **원어·정확한 표기** / ② **한국어 명칭**: LIKE 연산자와 와일드카드 문자 '%' (퍼센트), '_' (언더스코어)
- ④ **정확한 정의**: 값이 특정 문자열 패턴과 일치하는지 확인하는 연산자다.
  - **'%'**: **0개 이상의 문자열**과 일치하는지 확인 (글자 수 제한 없음, 아예 없어도 됨).
  - **'_'**: **정확히 단일 문자(1글자)**와 일치하는지 확인 (글자 수를 고정할 때 사용).
- ⑥ **실제 사용 상황** / ⑦ **적용 예시** / ⑧ **결과**:
  - WHERE LastName LIKE '%son'; → son으로 끝나는 모든 성씨 (Peterson, Johansson).
  - WHERE FirstName LIKE '___a'; (언더스코어 3개 + a) → **정확히 4글자이면서 a로 끝나는** 이름만 조회 (Kara, Emma, Puja).
  - WHERE name LIKE '_a%'; (확인문제 10번) → 첫 글자는 아무거나 1글자, **두 번째 글자가 'a'**이고 그 뒤에 0개 이상 문자가 오는 문자열.
- ⑨ **중요도 [★★★] · 혼동 주의점(반례)**: LIKE 'Lee%'는 'Lee'가 *포함된* 문자열이 아니라 **'Lee'로 시작하는** 문자열이다(포함 조건은 '%Lee%'). 또한 'Lee'(뒤에 0글자)도 'Lee%'에 일치한다!

- ⑩ **원본 출처**: db 1.pdf p.103~109, p.143 / db 2.pdf p.104, p.106, p.108

[TERM-11] NULL 비교와 SQL의 3값 논리 (Three-Valued Logic: IS NULL vs =)

- ① **원어·정확한 표기** / ② **한국어 명칭**: NULL / SQL 3값 논리 (TRUE, FALSE, UNKNOWN) / 명시적 비교 IS NULL, IS NOT NULL
- ③ **배경 및 맥락** / ④ **정확한 정의**: SQL에서 NULL은 숫자 0이나 빈 문자열('') 같은 실제 값이 아니라 **"값이 없음(부재)" 또는 "알 수 없음(불확실성)"**을 의미한다. 따라서 SQL은 참(TRUE), 거짓(FALSE) 외에 제3의 논리값인 **UNKNOWN(알 수 없음)**을 사용하는 **3값 논리 체계**로 동작한다.
- ⑤ **쉬운 설명** / ⑥ **사용 상황** / ⑦ **적용 예시** / ⑧ **결과**: 철수의 나이를 모르고(NULL) 영희의 나이도 모를 때(NULL), "철수와 영희의 나이가 같은가(NULL = NULL)?"라고 물으면 "같다(TRUE)"고 답할 수 없고 "모른다(UNKNOWN)"가 된다. WHERE 절은 조건 결과가 오직 TRUE인 행만 통과시키므로, WHERE Company = NULL을 실행하면 결과는 항상 **0건(빈 결과)**이 된다.

```
-- 올바른 NULL 및 Boolean 비교
SELECT LastName, FirstName, Company, Country FROM customers
WHERE Company IS NULL AND Country = 'USA';
```

- ⑨ **중요도 [★★★] · 핵심 구별점**: 일반 값의 동등성 비교에는 =를 쓰지만, NULL이나 Boolean(IS TRUE)처럼 특별한 상태를 비교할 때는 반드시 **IS / IS NOT**을 사용해야 한다.
- ⑩ **원본 출처**: db 1.pdf p.91~94, p.135~138

[TERM-12] LIMIT row_count OFFSET offset (조회 개수 제한 및 건너뛰기)

- ① **원어·정확한 표기** / ② **한국어 명칭**: LIMIT row_count [OFFSET offset];
- ④ **정확한 정의**:
  - row_count: 조회하여 화면에 가져올 최대 레코드 개수.
  - offset: 맨 앞에서부터 **건너뛸(제외할) 레코드 개수** (0부터 시작).
- ⑥ **실제 사용 상황** / ⑦ **적용 예시** / ⑧ **결과**: "tracks 테이블에서 Bytes 내림차순으로 **4번째부터 7번째 데이터만** 조회하시오."
  - "4번째부터" ⇒ 앞의 1, 2, 3번째(총 **3개**)를 건너뛰어야 하므로 **OFFSET 3**
  - "4번째~7번째" ⇒ 4, 5, 6, 7번째(총 7 - 4 + 1 = 4개)를 가져와야 하므로 **LIMIT 4**

```
SELECT TrackId, Name, Bytes FROM tracks ORDER BY Bytes DESC LIMIT 4 OFFSET 3;
```

- ⑨ **중요도 [★★★] · 혼동 주의점**: LIMIT 3, 4처럼 쉼표로 쓰는 축약형은 SQLite/MySQL에서만 동작하며 앞 숫자가 offset(3)이고 뒷 숫자가 row_count(4)라 순서가 반대여서 매우 헷갈리므로 원본 교재에서도 사용을 권장하지 않는다. 항상 명시적인 LIMIT N OFFSET M으로 기억하라.
- ⑩ **원본 출처**: db 1.pdf p.110~116

[TERM-13] GROUP BY & Aggregation Functions (집계 함수 5종)

- ① **원어·정확한 표기** / ② **한국어 명칭**: GROUP BY c1, c2, ... / Aggregation Functions (집계 함수: SUM, AVG, MAX, MIN, COUNT)
- ④ **정확한 정의**: FROM 및 WHERE 절 뒤에 배치되어 지정한 필드 값이 같은 레코드들을 하나의 그룹으로 묶어 요약본을 생성한다. 집계 함수는 각 그룹의 값들에 대해 계산을 수행하고 단일 요약값을 반환한다.
- ⑥ **실제 사용 상황** / ⑦ **적용 예시** / ⑧ **결과**:

```
SELECT Composer, AVG(Bytes) AS avgOfBytes
FROM tracks
GROUP BY Composer
ORDER BY avgOfBytes DESC;
```

결과: 작곡가(Composer)별로 트랙들을 묶은 뒤 각 그룹의 평균 Bytes를 구하고, SELECT에서 붙인 별칭(avgOfBytes)을 그다음 실행 단계인 ORDER BY에서 그대로 사용하여 내림차순 정렬한다. (참고: GROUP BY만 단독으로 쓰면 DISTINCT처럼 중복이

제거된 그룹 목록이 나온다.)

- ⑩ **원본 출처**: db 1.pdf p.118~125

[TERM-14] HAVING vs WHERE & SELECT 문 7단계 실행 순서

- ① **원어·정확한 표기** / ② **한국어 명칭**: HAVING condition (집계 항목에 대한 세부 조건절) 및 DQL 실행 순서
- ③ **배경 및 맥락 (왜 에러가 발생하는가?)**: 작곡가별 평균 재생 시간(분)이 10분 미만인 그룹만 보려고 다음과 같이 작성하면 즉시 에러가 발생한다:

  ```
  -- [에러 발생 코드] misuse of aggregate: AVG()
  SELECT Composer, AVG(Milliseconds / 60000) AS avgOfMinute
  FROM tracks
  WHERE avgOfMinute < 10
  GROUP BY Composer;
  ```

  원인: WHERE 절은 GROUP BY로 그룹을 묶고 AVG()를 계산하기 **이전에** 개별 행을 필터링하는 단계이므로, 아직 존재하지도 않는 집계 결과(AVG)나 SELECT 별칭(avgOfMinute)을 평가할 수 없기 때문이다!
- ④ **정확한 정의 (WHERE vs HAVING 및 7단계 실행 순서)**:
  - **WHERE**: FROM 이후, GROUP BY **이전에 개별 행(Row)**에 조건을 걸어 필터링한다.
  - **HAVING**: GROUP BY 및 집계 함수 적용 **이후에 그룹화된 결과(Group)**에 조건을 걸어 특정 그룹만 선택한다.
  - **SELECT 문 7단계 실제 실행 순서 (필수 암기)**:
    ① **FROM** (어떤 테이블에서) → ② **WHERE** (어떤 개별 행 조건으로) → ③ **GROUP BY** (어떻게 그룹화하고) → ④ **HAVING** (어떤 그룹을 걸러낼지) → ⑤ **SELECT** (어떤 데이터를 조회·별칭 부여할지) → ⑥ **ORDER BY** (어떤 순서로 정렬할지) → ⑦ **LIMIT** (몇 개만 보여줄지)
- ⑨ **중요도 [★★★] · 회상 질문**: 작성 순서(SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT)와 실행 순서의 가장 결정적인 차이는 무엇인가? (정답: SELECT는 맨 앞에 쓰지만 실제 실행은 5번째여서, WHERE에서는 SELECT 별칭을 못 쓰고 ORDER BY에서는 쓸 수 있다!)
- ⑩ **원본 출처**: db 1.pdf p.76, p.126~131, p.151

PART 3. 테이블 관리(DDL) · 데이터 조작(DML) · 다중 테이블 JOIN (db 2.pdf)

[TERM-15] CREATE TABLE & PRAGMA table_info()

- ① **원어·정확한 표기** / ② **한국어 명칭**: CREATE TABLE table_name (column_1 data_type constraints, ...); / PRAGMA table_info('table_name');
- ④ **정확한 정의** / ⑦ **적용 예시** / ⑧ **결과**: 새 테이블의 필드명, 데이터 타입, 제약 조건을 정의하여 생성한다. SQLite 고유 명령어인 PRAGMA table_info('테이블명');을 실행하면 테이블의 스키마 메타데이터인 cid(Column ID, 0부터 시작하는 컬럼 고유 정수 식별자), name, type, notnull(0 또는 1), dflt_value(기본값, 미지정 시 내부적으로 NULL), pk(기본키 여부 1/0)를 표로 확인할 수 있다.
- ⑩ **원본 출처**: db 2.pdf p.9~15, p.24, p.110

[TERM-16] SQLite 5대 데이터 타입 & 타입 선호도 (Type Affinity) · 날짜와 시간

- ① **원어·정확한 표기** / ② **한국어 명칭**: SQLite Data Types (NULL, INTEGER, REAL, TEXT, BLOB) & Type Affinity (타입 선호도)
- ④ **정확한 정의**:
  - **5대 데이터 타입**: NULL(값 없음), INTEGER(정수), REAL(부동 소수점 실수), TEXT(문자열), BLOB(이미지·동영상·문서 등 바이너리 데이터).

  - **타입 선호도(Type Affinity)**: 컬럼에 데이터 타입이 명시적으로 지정되지 않았거나 SQLite가 직접 지원하지 않는 타 DBMS 타입명(예: `VARCHAR(50)`, `DATETIME`)을 썼을 때, SQLite가 문자열 패턴 규칙에 따라 5가지 선호도(`INTEGER`, `TEXT`, `BLOB`, `REAL`, `NUMERIC`) 중 하나로 자동 추론하는 특성이다.
  - **날짜와 시간**: SQLite에는 별도의 날짜/시간 전용 스토리지 타입이 없다! 대신 날짜/시간 함수(`DATE()` 등)를 사용해 표기 형식에 따라 `TEXT`(ISO8601 문자열), `REAL`(율리우스일), `INTEGER`(유닉스 타임스탬프) 값으로 저장한다.

- ⑨ **중요도 [★★★] · 혼동 주의점**:
  - `VARCHAR(50)`이라고 선언해도 SQLite는 이를 `TEXT` 선호도로 처리하며, **길이 제한 50자를 검사하지 않는다!**
  - `INTEGER` 선호도 열에 숫자 모양의 문자열(`'123'`)을 넣으면 자동으로 숫자 `123`으로 변환하여 저장한다(단, 숫자로 바꿀 수 없는 문자열은 에러 없이 그대로 저장하는 유연성을 가진다).

- ⑩ **원본 출처**: `db 2.pdf` p.16, p.94~96, p.99~100, p.111

[TERM-17] Constraints (제약 조건) & AUTOINCREMENT · NOT NULL 설계 기준

- ① **원어·정확한 표기** / ② **한국어 명칭**: Constraints (`PRIMARY KEY`, `NOT NULL`, `FOREIGN KEY`) / `AUTOINCREMENT` 키워드
- ④ **정확한 정의**:
  - **Constraints(제약 조건)**: 테이블 필드에 적용되는 규칙 또는 제한 사항으로, **데이터의 무결성(Integrity)을 유지하고 데이터베이스의 일관성을 보장**한다.
  - **`AUTOINCREMENT`**: 주로 PK 필드에 적용되어 새 레코드에 대해 **항상 이전 최대값보다 큰 고유 정수값을 자동 할당**하는 키워드다.

- ⑥ **실제 사용 상황** / ⑦ **적용 예시** / ⑨ **중요도 [★★★] (시험 단골 함정 3가지)**:
  1. **`INTEGER` 표기 강제**: SQLite에서 `AUTOINCREMENT`는 오직 **`INTEGER PRIMARY KEY AUTOINCREMENT`**에만 사용할 수 있다. 만약 `INT PRIMARY KEY AUTOINCREMENT`나 `BIGINT`로 쓰면 즉시 에러(`AUTOINCREMENT is only allowed on an INTEGER PRIMARY KEY`)가 발생한다!
  2. **삭제된 번호 재사용 불가**: `AUTOINCREMENT`가 걸린 필드에서 마지막 행(예: 5번)을 삭제하더라도, 다음 새 레코드는 5번을 재사용하지 않고 이전 최대값 다음인 **6번**을 할당받는다.
  3. **`NOT NULL` 실무 기준**: 모든 컬럼에 무조건 `NOT NULL`을 써야 하는 것은 아니다("NO"). 문자열 컬럼은 빈 문자열(`''`)로 대체해 `NOT NULL`을 주로 쓰지만, 숫자나 날짜처럼 `0`이 실제 의미를 가지는 컬럼은 값이 없음을 뜻하는 `NULL`을 허용하는 것이 자연스럽다.

- ⑩ **원본 출처**: `db 2.pdf` p.17~19, p.97~98, p.109~110

[TERM-18] ALTER TABLE (4대 구문) & DROP TABLE

- ① **원어·정확한 표기** / ② **한국어 명칭**:
  1. 필드 추가: `ALTER TABLE table_name ADD COLUMN column_definition;`
  2. 필드 이름 변경: `ALTER TABLE table_name RENAME COLUMN current_name TO new_name;`
  3. 필드 삭제: `ALTER TABLE table_name DROP COLUMN column_name;`
  4. 테이블 이름 변경: `ALTER TABLE table_name RENAME TO new_table_name;`
  5. 테이블 자체 삭제: `DROP TABLE table_name;`

- ⑥ **실제 사용 상황** / ⑦ **적용 예시**:

```
ALTER TABLE examples ADD COLUMN Country VARCHAR(100) NOT NULL DEFAULT 'default value';
ALTER TABLE examples ADD COLUMN Age INTEGER NOT NULL DEFAULT 0;
```

- ⑨ **중요도 [★★★] · 혼동 주의점 · 반례**:
  - *필수 조건 1 (`NOT NULL` + `DEFAULT`)*: `CREATE TABLE` 시점에는 기본값이 없어도 `NOT NULL` 필드를 만들 수 있지만, 이미 생성된 테이블에 `ALTER TABLE ... ADD COLUMN`으로 **`NOT NULL` 제약조건이 있는 필드를 추가할 때는 기존 행들을 채울 `NULL`이 아닌 기본값(`DEFAULT ...`)을 반드시 함께 설정**해야 한다!

  - *필수 조건 2 (단일 문 다중 추가 불가)*: SQLite는 하나의 `ALTER TABLE` 문으로 여러 필드를 동시에 추가할 수 없다(필드마다 각각 실행해야 함).
  - *핵심 비교*: `ALTER TABLE`로 할 수 **없는** 작업은? → **전체 테이블 삭제**(이는 `DROP TABLE`로 수행한다. `db 2.pdf` 확인문제 5번, 12번).

- ⑩ **원본 출처**: `db 2.pdf` p.20~39, p.103~104, p.106

[TERM-19] INSERT / UPDATE / DELETE & 서브쿼리(Subquery) 응용 삭제

- ① **원어·정확한 표기** / ② **한국어 명칭**:
  - 삽입: `INSERT INTO table_name (c1, c2, ...) VALUES (v1, v2, ...), (v3, v4, ...);`
  - 수정: `UPDATE table_name SET c1 = v1, c2 = v2 [WHERE condition];`
  - 삭제: `DELETE FROM table_name [WHERE condition];`

- ⑥ **실제 사용 상황** / ⑦ **적용 예시 (서브쿼리를 결합한 조건부 삭제)**: "`articles` 테이블에서 작성일(`createdAt`)이 오래된 순으로 레코드 2개를 삭제하시오."

```
DELETE FROM articles
WHERE id IN (
    SELECT id FROM articles
    ORDER BY createdAt
    LIMIT 2
);
```

- ⑧ **결과 및 원리** / ⑨ **중요도 [★★★]**:
  - 괄호 안의 **서브쿼리(`SELECT id ... ORDER BY createdAt LIMIT 2`)가 먼저 실행**되어 가장 오래된 게시글 2개의 `id` 목록(`4, 3`)을 만들고, 바깥의 `DELETE`가 `WHERE id IN (4, 3)` 조건으로 해당 행들만 정확히 삭제한다.
  - *치명적 실수 주의*: `UPDATE`나 `DELETE` 문에서 **`WHERE` 절을 작성하지 않으면 테이블의 모든 레코드가 일괄 수정되거나 전부 삭제**된다!

- ⑩ **원본 출처**: `db 2.pdf` p.42~67, p.102~105

[TERM-20] INNER JOIN vs LEFT JOIN & `LEFT JOIN + IS NULL` 패턴

- ① **원어·정확한 표기** / ② **한국어 명칭**:
  - 내부 조인: `SELECT ... FROM table_a INNER JOIN table_b ON table_b.fk = table_a.pk;`
  - 왼쪽 외부 조인: `SELECT ... FROM table_a LEFT JOIN table_b ON table_b.fk = table_a.pk;`

- ③ **배경 및 맥락**: 데이터 중복과 수정 누락을 막기 위해 게시글(`articles`)과 회원(`users`) 테이블을 분리(정규화)하면 관리가 편해지지만, 화면에 "게시글 제목과 작성자 이름"을 함께 출력하려면 분리된 두 테이블을 다시 하나로 묶어야 한다. 이때 사용하는 것이 **`JOIN`**이다.

- ④ **정확한 정의** / ⑦ **적용 예시** / ⑧ **결과 비교 (`articles` 5건 중 4번 글은 `userId=4`로 `users(1,2,3번)`에 없는 회원, 3번 회원 '유하선'은 작성 글 없음)**:
  - **`INNER JOIN`**: 두 테이블에서 조인 조건(`ON`) 값이 **양쪽 모두 일치하는 레코드만** 반환한다. (4번 게시글은 `userId=4`가 `users`에 없으므로 제외되어 총 4건만 출력).
  - **`LEFT JOIN`**: `FROM` 절에 적은 **왼쪽 테이블의 모든 레코드를 무조건 유지**하고, 오른쪽 테이블에 매칭되는 값이 없으면 **`NULL`**로 채워 반환한다. (`articles LEFT JOIN users` 시 4번 게시글도 유지되며 `name`이 `NULL`로 출력되어 총 5건 출력).
  - **`LEFT JOIN + WHERE IS NULL` ("~한 적 없는" 안티조인 패턴)**:

```
-- 게시글을 작성한 이력이 한 번도 없는 회원 조회
SELECT users.name FROM users
LEFT JOIN articles ON articles.userId = users.id
WHERE articles.userId IS NULL;
```

결과: users를 왼쪽에 두어 모든 회원을 살린 뒤, 매칭된 게시글이 없어 articles.userId가 NULL이 된 **'유하선'**만 정확히 찾아낸다!

- ⑨ **중요도 [★★★] · SQLite 특수 주의점**: SQLite는 기본 설정에서 외래 키 제약조건 검사를 하지 않아(PRAGMA foreign_keys = OFF 기본), 부모 테이블(users)에 존재하지 않는 userId=4도 articles에 에러 없이 INSERT된다(다른 DBMS에서는 즉시 에러 발생).
- ⑩ **원본 출처**: db 2.pdf p.68~92, p.110

PART 4. Django Many-to-One Relationships 01 (db 3.pdf)

[TERM-21] 모델 관계의 종류 (1:1 / N:1 / N:M) & 외래 키(FK) 배치 원칙

- ① **원어·정확한 표기** / ② **한국어 명칭**: One-to-One (1:1) / Many-to-One (N:1, 다대일) / Many-to-Many (N:M, 다대다)
- ④ **정확한 정의**:
  - **1:1 관계**: 한 테이블의 레코드가 다른 테이블의 단 하나의 레코드와만 매칭 (예: 사용자 1명 ↔ 프로필 1개).
  - **N:1 (Many to One) 관계**: 한 테이블의 **0개 이상의 레코드(N)**가 다른 테이블의 **레코드 한 개(1)**와 관련된 관계 (예: 여러 댓글 Comment(N) → 하나의 게시글 Article(1), 주문(N) → 고객(1), 계좌(N) → 은행(1), 도시(N) → 국가(1)).
  - **N:M (Many to Many) 관계**: 양쪽 모두 여러 레코드와 연결되며 중개 테이블(Through Table)이 필요함 (예: 학생(N) ↔ 수업(M), 영화(N) ↔ 배우(M) — *주의: 영화와 배우는 N:1이 아님!*).
- ⑥ **실제 사용 상황** / ⑨ **중요도 [★★★] (왜 외래 키는 반드시 N쪽에 두는가?)**: 만약 1을 담당하는 Article 테이블에 외래 키(댓글 정보)를 두면, 1번 게시글에 댓글이 3개 달릴 때마다 **동일한 1번 게시글의 id, title, content가 매번 중복 저장되는 심각한 낭비와 무결성 파괴**가 발생한다. 따라서 외래 키(FK)는 반드시 **N을 담당하는 테이블(Comment, Article(대 User))**에 위치시켜 부모의 PK만 참조하게 해야 한다.
- ⑩ **원본 출처**: db 3.pdf p.8~15, p.73, p.75

[TERM-22] models.ForeignKey(to, on_delete) & Migration 필드 생성 규칙

- ① **원어·정확한 표기** / ② **한국어 명칭**: `article = models.ForeignKey(Article, on_delete=models.CASCADE)`
- ④ **정확한 정의**:
  - **to 속성 (필수 1)**: 참조하는 부모 모델 클래스 이름 (N:1에서 1의 클래스 정보).
  - **on_delete 속성 (필수 2)**: 참조하는 부모 객체(1)가 삭제될 때 자식 객체(N)를 어떻게 처리할지 정의하는 데이터 무결성 설정.
    - CASCADE: 부모 객체가 삭제되면 이를 참조하는 모든 자식 객체(댓글)도 함께 삭제.
    - PROTECT: 자식 객체가 하나라도 존재하면 부모 객체 삭제를 금지(에러 발생).
    - SET_NULL: 부모가 삭제되면 자식 필드에 NULL 저장 (단, null=True 설정 필수).
- ⑥ **실제 사용 상황** / ⑧ **Migration 결과** / ⑨ **중요도 [★★★]**:
  - **단수형 소문자 변수명 권장**: 단일 객체를 참조하므로 클래스 변수명을 article(소문자 단수형)로 작성하면, Migration 이후 실제 DB 테이블(articles_comment)에는 **'작성한 외래키 필드명' + '_' + 'id'** 규칙에 따라 **article_id**라는 컬럼이 생성된다(만약 변수명을 article_id로 지으면 article_id_id가 되어버림!).
  - **자료형 및 생성 위치**: 생성된 article_id 컬럼은 약 9경까지 저장 가능한 64비트 정수형인 **bigint** 자료형을 가지며, models.py 내 작성 순서와 무관하게 **테이블의 가장 마지막 필드**로 생성된다.
- ⑩ **원본 출처**: db 3.pdf p.17~21, p.64~65, p.74~75

[TERM-23] 참조 (Reference, N → 1) vs 역참조 (Reverse Reference, 1 → N)

- ① **원어·정확한 표기** / ② **한국어 명칭**:
  - **참조 (Reference)**: `comment.article` (N쪽 인스턴스에서 1쪽 인스턴스로 직접 접근)
  - **역참조 (Reverse Reference)**: `article.comment_set.all()` (1쪽 인스턴스에서 자신을 참조하는 N쪽 데이터들을 거꾸로 조회)
- ③ **배경 및 맥락** / ⑤ **쉬운 설명**: 댓글(Comment) 테이블에는 내가 몇 번 글의 댓글인지 적힌 `article_id` 칸이 있지만(내 휴대폰에 저장된 친구 번호 → **참조**), 게시글(Article) 테이블에는 댓글과 관련된 어떤 필드도 존재하지 않는다(내 번호를 저장한 사람들을 거꾸로 수소문해야 함 → **역참조**).
- ④ **정확한 정의 및 역참조 3단계 구조**: `article.comment_set.all()`은 ① **모델 인스턴스**(`article`: 1쪽 모델) + ② **related manager (역참조 이름)**(`comment_set`: FK를 정의한 모델명을 소문자로 바꾸고 `_set`을 붙여 Django가 자동 생성한 매니저) + ③ **QuerySet API**(`.all()`, `.filter()`, `.count()` 등)로 구성된다.
- ⑦ **적용 예시** / ⑨ **중요도 [★★★] (혼동 주의점 3가지)**:
  1. `comment.article`은 `<Article: Article object (1)>` 객체를 반환하고, `comment.article_id`(또는 `comment.article.pk`)는 숫자 `1`을 반환한다. 단, **`comment.article_pk`라는 필드는 존재하지 않아 에러가 난다!**
  2. 댓글 생성 시 `comment.article_id = 1`처럼 숫자를 직접 넣는 것보다 **`comment.article = article`처럼 객체를 직접 할당하는 것을 권장**한다(잘못된 모델 타입 할당 시 즉시 에러로 검증 가능하기 때문).
  3. *확인문제 9번 함정*: "ForeignKey로 연결된 모델은 역참조 시 자동으로 `related_name` 속성이 생성된다" → **거짓(X)**. `related_name`은 개발자가 직접 지정할 때 쓰는 옵션 이름이고, 지정하지 않았을 때 자동 생성되는 기본 역참조 매니저 형식은 **`소문자모델명_set`(예: `comment_set`)**이다!
- ⑩ **원본 출처**: `db 3.pdf` p.24~40, p.72~77

[TERM-24] CommentForm 외래 키 제외 & `save(commit=False)` · Variable Routing

- ① **원어·정확한 표기** / ② **한국어 명칭**: `fields = ('content',)` & `comment = comment_form.save(commit=False)`
- ③ **배경 및 맥락**: `CommentForm`에서 `fields = '__all__'`을 쓰면 댓글 입력창 옆에 게시글(`Article`)을 고르는 드롭다운 메뉴가 노출된다. 사용자가 엉뚱한 글을 고르거나 댓글 작성 금지 글에 댓글을 달 수 있으므로, `fields = ('content',)`로 외래 키 필드를 폼 출력에서 제외해야 한다(*주의: 단일 요소 튜플은 반드시 끝에 콤마 , 작성 필수!*).
- ④ **정확한 정의 (`save(commit=False)`)**: 폼에서 `article`을 제외했기 때문에 뷰 함수에서 바로 `comment_form.save()`를 호출하면 `NOT NULL constraint failed: articles_comment.article_id` 에러가 발생한다. **`save(commit=False)`는 데이터베이스에 즉시 저장 요청(INSERT)을 보내지 않고, 메모리상에 모델 인스턴스만 생성하여 반환**하는 메서드다.
- ⑥ **실제 사용 상황** / ⑦ **적용 예시** / ⑧ **결과**:

```
# articles/views.py (단일 책임 원칙에 따라 detail 뷰와 분리된 POST 전용 뷰)
def comments_create(request, pk):
    article = Article.objects.get(pk=pk) # Variable routing으로 받은 게시글 조회
    comment_form = CommentForm(request.POST)
    if comment_form.is_valid():
        comment = comment_form.save(commit=False) # 1) DB 저장 보류, 인스턴스만 생성
        comment.article = article                 # 2) 누락된 외래 키 객체 주입
        comment.save()                            # 3) 최종 DB 저장 요청
        return redirect('articles:detail', article.pk)
```

- ⑩ **원본 출처**: `db 3.pdf` p.43~54, p.74, p.77

[TERM-25] 댓글 READ / DELETE & DTL `{% for ... %}{% empty %}` · `length` / `count()`

- ① **원어·정확한 표기** / ② **한국어 명칭**:
  - 삭제 URL: `path('<int:article_pk>/comments/<int:comment_pk>/delete/', views.comments_delete, ...)`
  - 빈 목록 처리: `{% for comment in comments %} ... {% empty %} <p>댓글이 없어요..</p> {% endfor %}`

  - 개수 출력: {{ comments|length }}, {{ article.comment_set.all|length }}, {{ article.comment_set.count }}

- **④ 정확한 정의 및 핵심 포인트**:
  - detail 뷰에서 특정 글의 댓글만 가져오려면 Comment.objects.all()(전체 글의 댓글 조회 — 오답!)이 아니라 역참조인 **article.comment_set.all()**을 사용해야 한다.
  - 댓글 삭제 URL에 comment_pk뿐만 아니라 **article_pk까지 함께 받는 이유**는 URL 패턴의 일관성 유지 및 **댓글 삭제 후 다시 원래 게시글 상세 페이지(redirect('articles:detail', article_pk))로 돌아가기 위함**이다.
- **⑩ 원본 출처**: db 3.pdf p.55~62, p.68~70, p.73

PART 5. User N:1 관계 · 권한 제어 · View Decorators · ERD (db 4.pdf)

[TERM-26] settings.AUTH_USER_MODEL vs get_user_model() & 지연 평가 (Lazy Evaluation)

- **① 원어·정확한 표기 / ② 한국어 명칭**: User 모델을 참조하는 2가지 표준 방법과 지연 평가 원리
- **③ 배경 및 맥락**: Django에서는 기본 User 모델을 직접 import해서 사용하지 않는다. 만약 articles/models.py가 실행되는 시점에 accounts 앱의 User 클래스가 아직 메모리에 로딩되지 않았다면 참조 에러가 발생할 수 있기 때문이다.
- **④ 정확한 정의 (비교표)**:

구분 기준	settings.AUTH_USER_MODEL	get_user_model()
import 경로	from django.conf import settings	from django.contrib.auth import get_user_model
반환 값(Return Type)	'accounts.User' 형태의 **문자열(String)**	현재 활성화된 **User 모델 클래스(Class 객체)**
사용 위치	오직 **models.py 내부**에서 외래 키/관계 정의 시	**models.py를 제외한 다른 모든 위치** (views.py, forms.py 등)
동작 원리	앱 로딩 순서에 영향을 받지 않도록 문자열 경로만 등록해 두고, 모든 모델이 완전히 로딩된 후 실제 클래스를 가져와 연결하는 **지연 평가 (Lazy Evaluation)** 방식으로 동작	호출 시점에 이미 활성화된 User 모델 클래스를 즉시 평가하여 반환

- **⑥ Migration 시 주의점 (db 4.pdf p.10~12, 30~31)**: 이미 게시글이나 댓글 데이터가 존재하는 테이블에 user = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE)를 추가하고 makemigrations를 실행하면, 기존 행들의 NOT NULL 제약조건을 만족시키기 위해 기본값 입력 프롬프트가 뜬다. 이때 1(지금 직접 기본값 제공) 선택 후 다시 1(1번 회원 PK)을 입력하면 기존 글/댓글의 작성자가 모두 1번 회원(user_id = 1)으로 채워진다(단, DB에 1번 회원이 없으면 migrate 시 에러 발생!).
- **⑩ 원본 출처**: db 4.pdf p.8~12, p.29~31, p.67, p.70~72

[TERM-27] 작성자 정보 저장(request.user) & 수정·삭제 권한 이중 제어

- **① 원어·정확한 표기 / ② 한국어 명칭**: article.user = request.user / if request.user == article.user: (또는 comment.user)
- **③ 배경 및 맥락**: 게시글/댓글 작성 시 User 선택 창을 폼에 그대로 노출하면 다른 사람을 선택해 글을 올리는 **사칭 문제**가 발생한다. 따라서 ArticleForm에서도 fields = ('title', 'content',)로 user를 제외하고, 뷰 함수에서 현재 로그인한 사용자 객체인 **request.user**를 주입해야 한다.
- **④ 정확한 정의 및 이중 방어(UI + View) 원칙**:

1. **생성(CREATE)**: `article = form.save(commit=False)` → **`article.user = request.user`** → `article.save()` (댓글도 동일하게 `comment.user = request.user` 추가).
2. **템플릿 UI 방어**: `{% if request.user == article.user %}` (댓글은 `comment.user`) 조건문으로 감싸 작성자 본인에게만 UPDATE/DELETE 버튼을 보여준다.
3. **서버 View 방어**: 템플릿에서 버튼을 숨겼더라도 **requests 패키지나 Postman 같은 툴을 이용해 브라우저를 거치지 않고 직접 삭제/수정 URL로 POST 요청을 보낼 수 있으므로**, 반드시 `views.py`의 `update`, `delete`, `comments_delete` 함수 내부에서도 **`if request.user == article.user`**: (또는 `comment.user`)를 다시 검사해야 한다!

- ⑩ **원본 출처**: `db 4.pdf` p.14~26, p.33~40, p.67~72

[TERM-28] View Decorators & Allowed HTTP Methods (`405 Method Not Allowed`)

- ① **원어·정확한 표기** / ② **한국어 명칭**:
  - 인증 제한: `@login_required`
  - 지정 메서드만 허용: `@require_http_methods(["GET", "POST"])` (인자는 대문자 문자열 리스트)
  - 조회(안전) 메서드만 허용: `@require_safe` (`GET`과 `HEAD`만 허용)
  - POST 메서드만 허용: `@require_POST`
- ④ **정확한 정의** / ⑧ **위반 시 결과**: View 함수 위에 붙여서 코드 흐름을 방해하지 않고 사전 체크리스트처럼 접근 조건을 검사하는 Python 데코레이터다. 허용되지 않은 HTTP method로 요청하면 **`HttpResponseNotAllowed` — 상태 코드 `405` (`Method Not Allowed`) 오류**를 반환한다.
- ⑨ **중요도 [★★★] · 왜 `@require_GET` 대신 `@require_safe`를 권장하는가?**: `GET`과 `HEAD`는 서버 상태를 변경하지 않고 조회만 수행하는 **safe method**로 정의되어 있다. 웹 클라이언트나 브라우저는 리소스 본문 없이 메타데이터만 확인할 때 `HEAD` 요청을 보내는데, `@require_GET`을 쓰면 `HEAD` 요청까지 차단해버리므로 더 포괄적이고 표준 호환성이 높은 **`@require_safe`**를 권장한다.
- ⑩ **원본 출처**: `db 4.pdf` p.41~49, p.65, p.68

[TERM-29 & TERM-30] ERD 3대 구성요소 & Cardinality (까마귀 발 표기법 6종)

- ① **원어·정확한 표기** / ② **한국어 명칭**: ERD (Entity-Relationship Diagram, 개체-관계 다이어그램) / Cardinality (카디널리티: 수적 관계) & Crow's foot notation (까마귀 발 표기법)
- ④ **정확한 정의**:
  - **Entity (개체)**: 데이터베이스에 저장되는 객체나 개념으로 **테이블(Table)**로 표현됨 (예: `User`, `Article`, `Comment`).
  - **Attribute (속성)**: 엔티티가 가지는 고유한 데이터 항목으로 **컬럼(Column/Field)**으로 표현됨 (예: `id`, `title`, `content`).
  - **Relationship (관계)**: 엔티티 간의 연관성으로 테이블 간 연결된 **선(외래 키 FK 참조선)**으로 표현됨.
- ⑥ **Cardinality 6종 기호 판독 공식 (`db 4.pdf` p.58 완벽 정리)**: 선 끝부분에 붙는 3가지 기본 기호—**동그라미 0 (Zero, 0개 가능/선택)**, **수직선 | (One, 정확히 1개/필수)**, **삼발이 < (Many, 다수)**—를 조합하여 읽는다.
  1. **Zero or one (0 또는 1)**: 동그라미(0) + 수직선(|) → —0|—
  2. **One (1)**: 수직선 1개(|) → ——|——
  3. **One (and only one) (오직 1개)**: 수직선 2개(||) → —||——
  4. **Many (다수)**: 까마귀 발(<) 단독 → ——<
  5. **Zero or many (0개 이상 다수)**: 동그라미(0) + 까마귀 발(<) → —0< (예: 회원은 댓글을 0개 이상 작성한다 → `Comment` 쪽 끝에 표시)
  6. **One or many (1개 이상 다수)**: 수직선(|) + 까마귀 발(<) → —|<
- ⑩ **원본 출처**: `db 4.pdf` p.50~62, p.68~70

PART 6. 헷갈리는 핵심 용어·개념 쌍 결정적 차이 비교표

시험과 실무에서 가장 빈번하게 섞여 출제되는 6쌍의 유사 개념을 **결과를 바꾸는 결정적 조건** 중심으로 대조하였다.

비교 대상 쌍	개념 A (특징 및 작동 조건)	개념 B (특징 및 작동 조건)	출처
WHERE vs **HAVING**	**WHERE**: `GROUP BY` **이전** 실행. 개별 행(Row) 필터링. 집계 함수(`AVG`, `COUNT` 등) 사용 시 `misuse of aggregate` 에러 발생.	**HAVING**: `GROUP BY` 및 집계 **이후** 실행. 그룹화된 요약 결과에 조건 지정. 집계 함수 조건식 사용 가능.	db 1 p.129
= (등호) vs **IS 연산자**	**=**: 일반 숫자·문자열 동등 비교. 피연산자 중 하나라도 `NULL`이면 결과가 `UNKNOWN`이 되어 0건 조회됨.	**IS / IS NOT**: `NULL` 여부(`IS NULL`) 및 Boolean(`IS TRUE`)을 명시적으로 판별할 때 사용하는 표준 구문.	db 1 p.138
DROP TABLE vs **DELETE FROM**	**DROP TABLE (DDL)**: 테이블의 스키마(구조)와 데이터를 통째로 완전히 철거·삭제함.	**DELETE FROM (DML)**: 테이블 구조(틀)는 그대로 두고 내부의 레코드(행)만 조건부 또는 전체 삭제함.	db 2 p.39, 62
INNER JOIN vs **LEFT JOIN**	**INNER JOIN**: 두 테이블에서 조인 키가 **양쪽 모두 존재하는 교집합 행만** 결과로 반환(고아 행/미작성자 탈락).	**LEFT JOIN**: **왼쪽 테이블의 모든 행을 무조건 보존**하고, 오른쪽 매칭 실패 시 해당 컬럼들을 `NULL`로 채움.	db 2 p.81, 87
참조(Reference) vs **역참조(Reverse)**	**참조(N→1)**: FK 필드를 가진 모델(`Comment`)에서 부모 조회(`comment.article`). 단일 객체 반환.	**역참조(1→N)**: FK 필드가 없는 모델(`Article`)에서 자식 목록 조회(`article.comment_set.all()`). QuerySet 반환.	db 3 p.34, 38
AUTH_USER_MODEL vs **get_user_model()**	**`settings.AUTH_USER_MODEL`**: 문자열(`'accounts.User'`) 반환. 지연 평가를 위해 **`models.py` 내부**에서만 사용.	**`get_user_model()`**: 활성화된 `User` 모델 클래스 반환. **`models.py` 이외의 모든 파일**에서 사용.	db 4 p.9

PART 7. 용어·핵심 지식 완전 정복 확인 문제 (6대 유형 12문항 & 상세 해설)

단순 뜻 연결을 넘어 ① 정의 보고 용어 쓰기, ② 상황별 개념 선택, ③ 유사 용어 구별, ④ 잘못 사용한 용어·코드 찾기, ⑤ 공식·문법 재현, ⑥ 응용 상황 판단의 6가지 유형을 결합한 실전 점검 문제다.

1. **[유형 1: 정의를 보고 정확한 용어 쓰기]** 관계형 데이터베이스에서 (A)는 고유한 데이터 타입이 지정되는 테이블의 세로 방향 항목(Attribute)을 뜻하고, (B)는 구체적인 데이터 값들이 저장되는 가로 한 줄(Tuple)을 뜻한다. 또한 SQLite에서 테이블의 `cid`, `notnull`, `pk` 등 스키마 메타데이터 구조를 확인하는 명령어는 (C)이다.
2. **[유형 2: 상황을 보고 필요한 개념·옵션 선택]** 사내 게시판에서 게시글(`Article`)이 삭제될 때 해당 게시글에 달린 댓글(`Comment`)은 삭제되지 않도록 막아 데이터 유실을 방지하려 한다. `models.ForeignKey(Article, on_delete=models._____)` 빈칸에 들어갈 가장 적절한 `on_delete` 옵션과, 반대로 부모 삭제 시 댓글의 외래 키 값을 `NULL`로 바꾸는 옵션(단, `null=True` 필요)을 각각 쓰시오.
3. **[유형 3: 서로 비슷한 용어·구문 구별]** Django 프로젝트의 `articles/models.py`에서 `Article` 모델에 작성자 외래 키를 정의할 때 사용하는 구문 (A)와, `articles/forms.py`나 `views.py`에서 현재 활성화된 User 모델 클래스를 가져올 때 사용하는 함수 (B)를 각각 정확히 구분하여 쓰시오. 왜 `models.py`에서는 (A)를 써야 하는지 동작 원리 용어를 포함해 설명하시오.
4. **[유형 4: 잘못 사용한 용어·문법 찾기 (오류 교정)]** 다음 4개의 SQL/Django 문장 중 문법 에러가 발생하거나 의도대로 동작하지 않는 **잘못된 부분**을 찾아 바르게 고치시오.
   - (가) `CREATE TABLE items (id INT PRIMARY KEY AUTOINCREMENT, name TEXT);`

- (나) `SELECT * FROM customers WHERE Company = NULL;`
- (다) `comment = Comment.objects.get(pk=1)` 이후 게시글 PK를 출력하려고 `print(comment.article_pk)` 실행
- (라) `class CommentForm(forms.ModelForm): class Meta: model = Comment; fields = ('content')`

5. **[유형 5: 공식·문법·규칙 재현]** `tracks` 테이블에서 재생 시간(`Milliseconds`)을 60,000으로 나누어 **소수점까지 살아 있는** 분 단위 값을 '재생 시간(분)'이라는 별칭으로 출력하되, 재생 시간이 긴 순서(내림차순)로 정렬하여 **6번째부터 10번째까지의 트랙(총 5개)**만 조회하는 단일 SQL 문을 작성하시오.
6. **[유형 6: 응용 상황 판단 (실행 순서 & 집계)]** `employees` 테이블에서 직무(`Title`)가 `'IT Staff'`가 아닌 직원들을 대상으로 부서(`Department`)별 인원수를 구한 뒤, 부서 인원수가 3명 이상인 부서만 인원수 내림차순으로 조회하려 한다. 이때 `'IT Staff'` 제외 조건과 `3명 이상` 조건은 각각 `WHERE`와 `HAVING` 중 어디에 작성해야 하며, 그 이유는 무엇인가?
7. **[유형 2: 상황별 JOIN 패턴 선택]** `users` 테이블과 `orders`(외래 키 `userId`) 테이블이 있을 때, 회원가입 후 **단 한 번도 주문을 하지 않은 회원의 이름**만 조회하는 SQL 쿼리 패턴을 작성하시오.
8. **[유형 3: 유사 와일드카드 구별]** `WHERE code LIKE 'A_0%'` 조건과 일치하는 값을 다음 보기에서 모두 고르시오: `['A0', 'A10', 'AB0', 'AB099', 'ABC0']`
9. **[유형 4: 잘못된 테이블 변경(DDL) 수정]** 이미 100명의 회원 데이터가 들어 있는 `users` 테이블에 `ALTER TABLE users ADD COLUMN grade INTEGER NOT NULL;`을 실행했더니 에러가 발생했다. 원인과 해결 쿼리를 쓰시오.
10. **[유형 5: Django 댓글 생성 로직 재현]** `comments_create(request, pk)` 뷰 함수에서 `comment_form.is_valid()` 통과 후, 게시글(`article`)과 로그인 작성자(`request.user`)를 모두 연결하여 DB에 저장하는 4줄의 Python 코드를 작성하시오.
11. **[유형 6: 보안 및 HTTP 메서드 판단]** 게시글 삭제 뷰 함수 `delete(request, pk)`에 ① 비로그인 사용자의 접근을 차단하고, ② 오직 `POST` 요청만 허용하며(GET 요청 시 405 반환), ③ 작성자 본인만 삭제할 수 있도록 검증하는 데코레이터 2개와 조건문 1개를 쓰시오.
12. **[유형 1 & 3: ERD Crow's foot 기호 판독]** ERD에서 `User` 엔티티와 `Article` 엔티티를 잇는 관계선의 `User` 쪽 끝에는 수직선 두 개(`||`)가 그려져 있고, `Article` 쪽 끝에는 동그라미와 삼발이(`0<`)가 그려져 있다. 두 기호의 정확한 영문 명칭과 비즈니스적 의미를 설명하시오.

정답 및 판단 근거 해설

문항	정답 (정확한 표기)	판단 근거 및 오답 함정 해설
1번	A: `Field` (또는 `Column`) B: `Record` (또는 `Row`) C: `PRAGMA table_info('테이블명');`	열(세로)은 데이터 타입을 정의하는 Field(Column/Attribute)이고, 행(가로)은 실제 값의 묶음인 Record(Row/Tuple)이다. SQLite 스키마 확인은 `PRAGMA table_info()`를 사용한다.
2번	삭제 방지: `PROTECT` NULL로 변경: `SET_NULL`	`CASCADE`는 부모와 함께 자식도 삭제하므로, 자식 존재 시 부모 삭제를 막으려면 `PROTECT`를 써야 한다. `SET_NULL`은 해당 ForeignKey 필드에 `null=True`가 함께 설정되어 있어야 동작한다.
3번	A: `settings.AUTH_USER_MODEL` B: `get_user_model()` 원리: **지연 평가(Lazy Evaluation)**	`models.py` 실행 시점에는 `User` 모델이 아직 로딩되지 않았을 수 있으므로 문자열(`'accounts.User'`)을 넣어 모든 앱 로딩 후 평가(지연 평가)되게 하고, 나머지 파일에서는 클래스 객체를 반환하는 `get_user_model()`을 호출한다.
4번	(가) `INT` → `INTEGER` (나) `= NULL` → `IS NULL` (다) `article_pk` → `article_id` (또는 `article.pk`) (라) `('content')` → `('content',)`	(가) SQLite의 `AUTOINCREMENT`는 `INTEGER PRIMARY KEY`에만 허용된다. (나) 3값 논리로 인해 `= NULL`은 `UNKNOWN`이 된다. (다) DB 생성 컬럼명은 `article_id`다. (라) 단일 요소 튜플은 끝에 콤마(`,`)가 없으면 단순 문자열로 인식되어 에러가 난다.

문항	정답 (정확한 표기)	판단 근거 및 오답 함정 해설
5번	`SELECT Milliseconds / 60000.0 AS '재생 시간(분)' FROM tracks ORDER BY Milliseconds DESC LIMIT 5 OFFSET 5;`	소수점을 유지하려면 `60000.0`으로 나누어야 한다. "6번째부터 10번째"는 앞의 1~5번째(5개)를 건너뛰므로 `OFFSET 5`이고, 6·7·8·9·10번째 총 5개를 가져오므로 `LIMIT 5`이다.
6번	`Title != 'IT Staff'` → **WHERE** `COUNT(*) >= 3` → **HAVING**	DQL 실행 순서상 `WHERE`는 그룹화 이전에 개별 직원의 직무를 걸러내고, `HAVING`은 `GROUP BY Department`로 묶은 뒤 집계된 그룹별 인원수(`COUNT(*)`)를 필터링한다.
7번	`SELECT users.name FROM users LEFT JOIN orders ON orders.userId = users.id WHERE orders.userId IS NULL;`	`users`를 왼쪽 테이블로 두고 `LEFT JOIN`하면 주문 이력이 없는 회원도 결과에 남고 오른쪽 `orders.userId`가 `NULL`이 된다. 이를 `WHERE orders.userId IS NULL`로 추출한다.
8번	**`'A10'`, `'AB0'`, `'AB099'`** (총 3개)	`_`는 정확히 1글자, `%`는 0글자 이상이다. `'A0'`는 A와 0 사이에 글자가 없어 탈락, `'ABC0'`는 A와 0 사이에 2글자(BC)가 있어 탈락한다.
9번	`ALTER TABLE users ADD COLUMN grade INTEGER NOT NULL DEFAULT 1;` (기본값 지정)	기존 레코드가 있는 테이블에 `NOT NULL` 컬럼을 추가할 때는 기존 행들에 채워 넣을 `NULL`이 아닌 기본값(`DEFAULT`)을 반드시 지정해야 한다.

10번	`comment = comment_form.save(commit=False)`
	`comment.article = article`
	`comment.user = request.user`
	`comment.save()`
		`save(commit=False)`로 DB 저장을 잠시 보류하고 인스턴스를 받아, 폼에서 제외했던 두 외래 키(`article`, `user`)를 주입한 뒤 `comment.save()`를 호출한다.

11번	데코레이터: `@login_required`, `@require_POST` 조건문: `if request.user == article.user:`	데코레이터로 인증 여부와 HTTP 메서드를 사전 차단하고, 함수 내부에서 요청자(`request.user`)와 작성자(`article.user`)의 일치 여부를 검증하여 권한 없는 삭제를 막는다.
12번	`||`: **One (and only one)** `0<`: **Zero or many**	각 게시글은 반드시 오직 1명의 작성자(User)에게만 속하며(`||`), 한 명의 회원은 게시글을 0개(안 쓸 수도 있음) 이상 여러 개 작성할 수 있다(`0<`)는 뜻이다.